

並列計算 GPU

本日の内容

GPUとCUDA

CEDの場合

CUDAプログラミング

GPU (Graphics Processing Unit)

元々は動画などのマルチメディアを用いたアプリケーション向け

→並列計算(含:AI学習・推論)にも使用

“nVIDIA”ではない (NVIDIA_Logo_Guidelines)

NVIDIA: Hopper, Blackwell, RTX

AMD: Instinct, Radeon

Intel: Data Center GPU MAX, Arc

高いFLOPSと低いB/Fが特徴



Byte per FLOP(B/F)

1 演算あたり何バイト読み書きできるか？

```
double a [ N ], b [ N ], c [ N ];  
:  
for ( int i = 0; i < N; i++ ) {  
    a [ i ] = b [ i ] + c [ i ];  
}
```

doubleを2個読んで1個書く→24バイト

足し算1回

$B/F = 24$ なら最高速度

現代的な計算機は低い傾向

A64FX: $1.0 \text{ TB/s} / 2.7 \text{ TFLOPS} = 0.37$
(0.5だった気もする)

A100: $1.6 \text{ TB/s} / 9.7 \text{ TFLOPS} = 0.16$

メモリが遅いのではなく演算性能が高すぎる

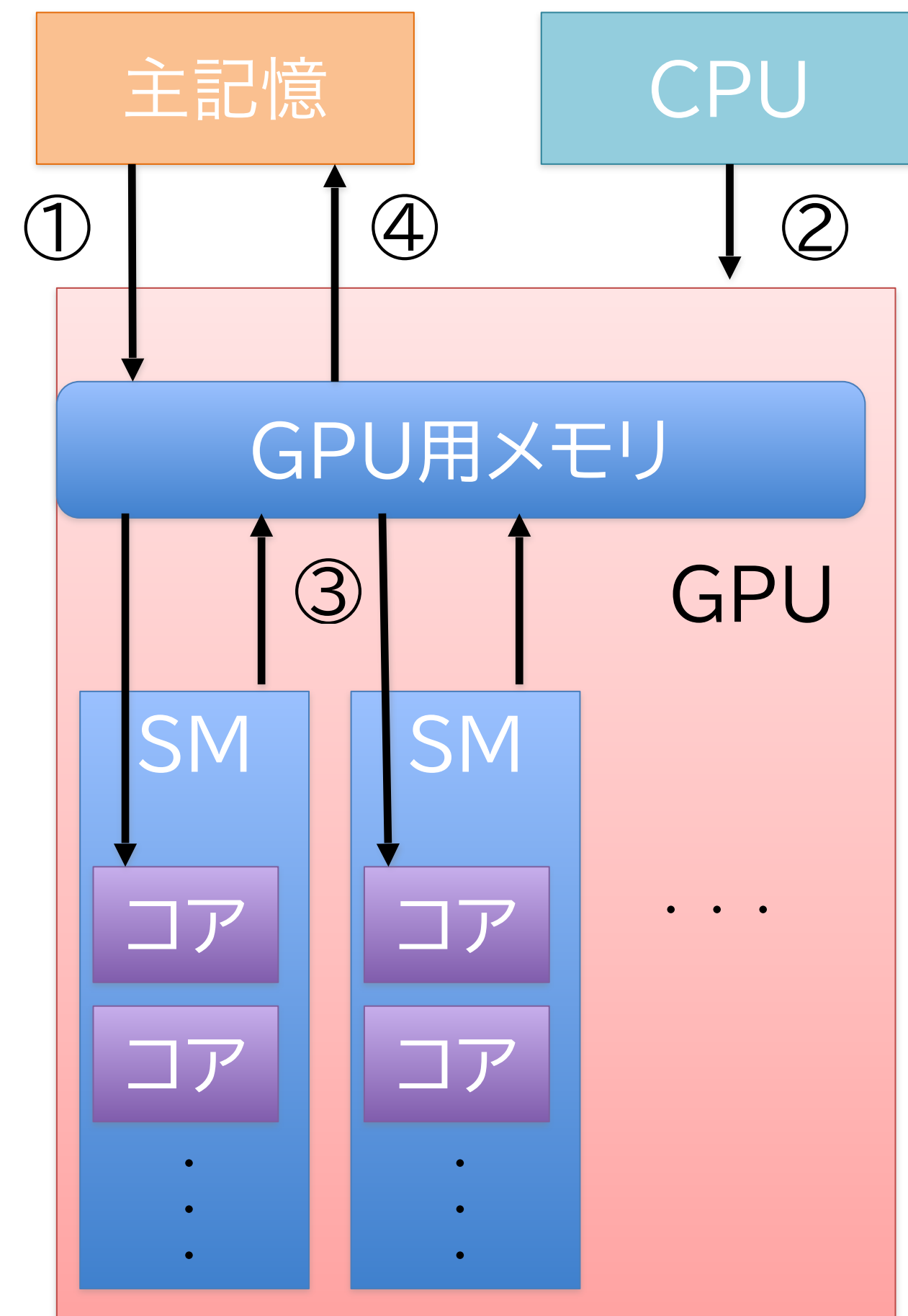
キャッシュメモリに載るのが前提

A64FXならL1 キャッシュにのれば $B/F = 4$

キャッシュメモリに乗り続けるように問題を分割し
たり計算の順序を入れ替える

反転させてFLOP per byteという時も

CUDA (Compute Unified Device Architecture)



SM: Streaming Multiprocessor

コア: CUDAコア (かつてのSP (Streaming Processor))

NVIDIAが提供するGPU向けの統合開発環境

対応言語

C++: C言語も利用可能

Fortran

並列モデル

分散メモリ型 (CPUとGPUで独立したメモリ)

SIMT (Single Instruction Multiple Threads)

処理の流れ

1. 主記憶からデータをGPU用メモリにコピー
2. CPUがGPUに対して処理を指示する
3. GPUが必要なデータを読み込み各コアで並列処理
4. 結果をGPU用メモリから主記憶にコピー

計算の一部を他所に流す(オフロード)

CEDで利用可能なGPU

NVIDIA T400

総コア数: 384 (6 SM x 64 コア)

メモリ: 3896 MB

単精度演算: 1.094 TFLOPS

全ての学生用端末



NVIDIA RTX A4000

総コア数: 6144 (48 SM x 128 コア)

メモリ: 16012 MB

単精度演算: 19.17 TFLOPS

gpu01等にsshして利用



アルファベットはアーキテクチャの世代 T: Turing; A: Ampere; H: Hopper; B: Blackwell, ...

CUDAプログラミング

GPUの制御はCPUから行なう → CPU上で実行するプログラムでGPUを制御

通常のC言語のプログラムの拡張

CUDAのプログラム:

通常ルーチン

CPUの処理

GPUの制御

GPUの制御ルーチン

GPU上での計算

CPU側(**ホスト**)とGPU側(**デバイス**)で異なるメモリを使用

互いのメモリ上のデータは直接参照ができない

→ **データを参照するためには転送が必要**

課題説明

課題1: CUDAプログラミングの基礎

- GPUでの計算

 - メモリの扱い

 - 関数(カーネル)呼び出し

 - ループ処理

 - ブロックとスレッド

- コンパイルの方法と実行

- 計算時間の測定

課題2: 銀河シミュレーションの準備

- 高速化

課題3: 銀河シミュレーション

課題1

加法定理をGPUで計算:

$$\sin(a+b) = \sin(a)\cos(b) + \cos(a)\sin(b)$$

C言語で書く場合:

```
void add(const int n, float *A, float *B, float *C){  
    for(int i = 0; i < n; i++){  
        C[i] = sin(A[i]) * cos (B[i]) + cos(A[i]) * sin(B[i]);  
    }  
}
```

GPUでの計算:

- メモリの扱い
- ループ処理
- 関数呼び出し
- ブロックとスレッド

課題1: CUDAプログラミングの基礎

ホストーデバイス間のデータ転送とデバイスで計算するプログラムの例

[illegible]

課題1: メモリの扱い

ホストメモリ(普通にmalloc, freeでも良い)とデバイスメモリの確保

```
float *ex_h, *ex_d;
cudaMallocHost(&ex_h, sizeof(float)*n));           // ホスト側のメモリの用意
memset(ex_h, 0, sizeof(float)*n);                 // 0で初期化
cudaFreeHost(ex_h);                                // メモリの解放

cudaMalloc(&ex_d, sizeof(float)*n));               // デバイス側のメモリの用意
cudaMemset(ex_d, 0, sizeof(float)*n));             // 0で初期化
cudaFree(ex_d);                                    // メモリの解放
```

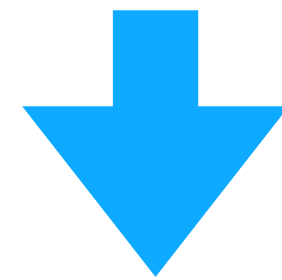
データのコピー

```
// ホスト側からデバイス側へコピー
cudaMemcpy(ex_d, ex_h, sizeof(float)*n, cudaMemcpyHostToDevice);
// デバイス側からホスト側へコピー
cudaMemcpy(ex_h, ex_d, sizeof(float)*n, cudaMemcpyDeviceToHost);
```


課題1: ループ処理

CPUにおけるループ処理

```
for(i = 0; i < n; i++) { A[i] = B[i] + C[i]; }
```



GPUはn個の処理(スレッド)として複数コアで並列処理

```
スレッド 0: A[0] = B[0] + C[0]  
スレッド 1: A[1] = B[1] + C[1]  
スレッド 2: A[2] = B[2] + C[2]  
スレッド 3: A[3] = B[3] + C[3]  
.....
```

1つの関数として設定

課題1：関数(カーネル)呼び出し

カーネル関数: GPUでの処理を記述した関数(`__global__` 必要)

ブロックIDとスレッドIDと呼ばれる組み込み変数を用いてデータ参照

```
__global__ void kernel_A(float *A, float *B, float *C){  
    int i = threadIdx.x + blockIdx.x * blockDim.x;  
    A[i] = B[i] + C[i];  
}
```

カーネル関数の呼び出し方(ブロックとスレッドはこの後説明)

```
dim3 blocks = dim3(nblocks, 1); // ブロック数の設定  
dim3 threads = dim3(nthreads, 1); // スレッド数の設定  
kernel_A<<<blocks, threads>>>(A, B, C);
```

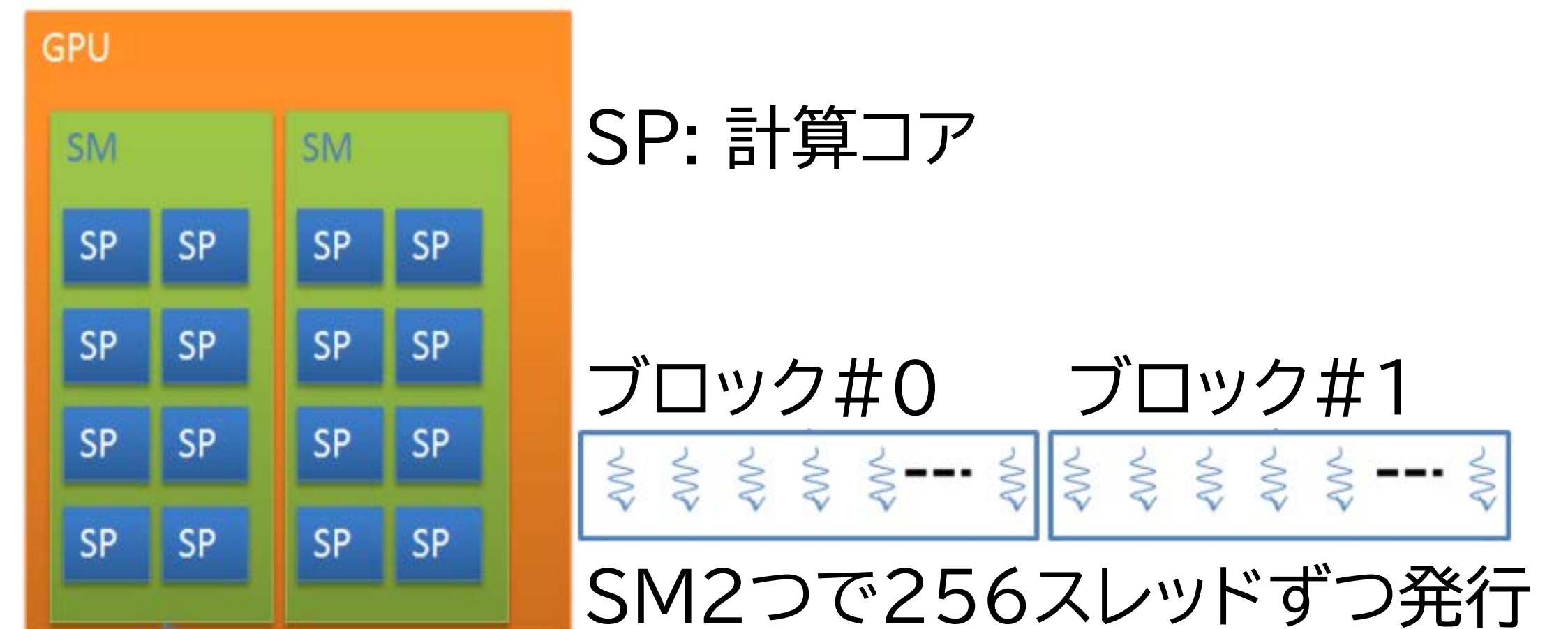

課題1: ブロックとスレッド

GPUは複数のマルチプロセッサ(SM)からなり、各SMは複数の計算コアを持つ
各SMにブロックを割り当て、各コアにスレッドを割り当てる

`kernel<<<1, 512>>>(…)` の場合



`kernel<<<2, 512>>>(…)` の場合



ブロックに対してSMが足りない場合、1つのSMが複数のブロックを順次処理
適切なブロック数とスレッド数を設定することにより、処理が高速になる

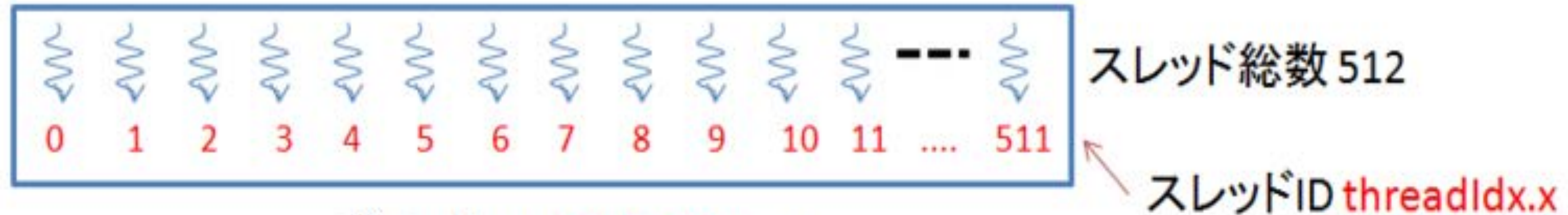
課題1: ブロックIDとスレッドID #1

カーネル関数の内容は1スレッドの処理内容

各スレッドで異なる処理を実施したい→**ブロックID**と**スレッドID**を利用

```
kernel_func<<<1,512>>>(...)
```

スレッドブロック数1 スレッドブロック内スレッド数 512を指定



ブロックID `blockIdx.x = 0`

スレッドブロック内スレッド数 `blockDim.x = 512`

スレッドブロック数 `gridDim.x = 1`

参照例: `int i = threadIdx.x + blockIdx.x * blockDim.x`

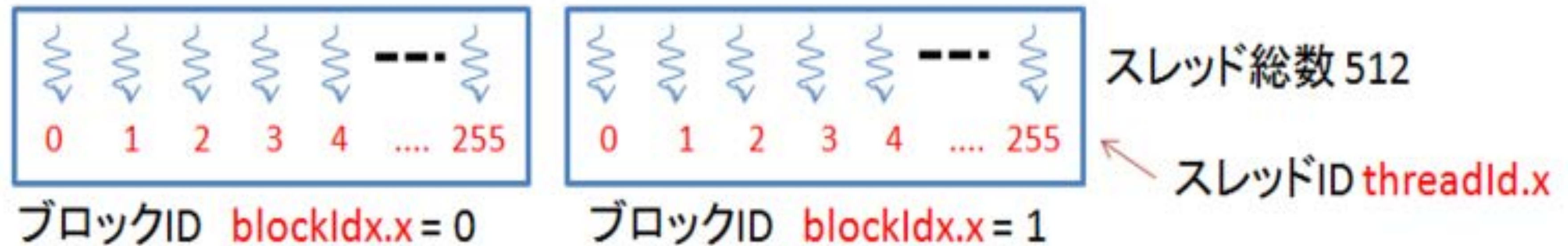
課題1: ブロックIDとスレッドID #2

カーネル関数の内容は1スレッドの処理内容

各スレッドで異なる処理を実施したい→**ブロックID**と**スレッドID**を利用

```
kernel_func<<<2,256>>>(...)
```

スレッドブロック数 2 スレッドブロック内スレッド数 256を指定



スレッドブロック内スレッド数 `blockDim.x = 256`

スレッドブロック数 `gridDim.x = 2`

参照例: `int i = threadIdx.x + blockIdx.x * blockDim.x`

課題1：ブロックIDとスレッドID #3

扱えるブロック数とスレッド数には上限がありGPUの種類ごとに違う

deviceQueryで確認

課題1:コンパイルと実行

コンパイル

サンプルプログラム: ex.cu

```
nvcc -O3 -o ex ex.cu
```

実行

```
./ex
```


課題1：計算時間の計測

カーネル(GPUで計算を行なっている部分)の計算時間を測定する

```
cudaEvent_t start, stop;
float time;
cudaEventCreate(&start);
cudaEventCreate(&stop);

cudaEventRecord(start, 0);
kernel_A<<<grid, threads >>> (ex_d, size_x, size_y);
cudaEventRecord(stop, 0);
cudaEventSynchronize(stop);
cudaEventElapsedTime ( &time, start, stop );
printf("%15.7f ", time); // 変数 time にstartとstopの時間差が入る(ミリ秒)

cudaEventDestroy( start );
cudaEventDestroy( stop);
```

課題1

加法定理をGPUで計算

ホストメモリで乱数を生成、デバイスメモリへ転送

デバイスで計算

デバイスメモリからホストメモリへ転送

スレッド数・ブロック数の変更

スレッド数・ブロック数を変化させ、計算時間を計測

最適なスレッド数とブロック数を調査する

計算精度

CPU(倍精度計算)とGPU(単精度計算)の結果を比較

課題2: 銀河シミュレーションの準備

銀河シミュレーションと同様のデータアクセスをする簡単なプログラムを作成し、高速化技法の効果を調べる。

乱数を生成し総和を求める

ホストメモリで乱数を生成し、デバイスメモリに転送

デバイスで総和計算

デバイスメモリからホストメモリへ転送

高速化

スレッド・ブロック数の設定やメモリアクセス回数削減による影響の調査

課題2: 銀河シミュレーションの準備

全てのスレッドで全てのデータを参照し、加算

```
__global__ void kernel_B(const int n, float *ex_d, float *add_d){
    int i = threadIdx.x + blockIdx.x * blockDim.x;
    add_d[i] = ex_d[i];
    for(int j = 0; j < n; j++) {
        if(i != j) {
            add_d[i] += ex_d[j]; // 全データを加算
        }
    }
}
```

高速化効果の確認

最適なブロック数・スレッド数の設定
メモリアクセス回数を減らす
分岐の削減 etc...

カーネルの中でif文を書いてはいけません 性能が下がる

```
void add_ALL(int n, float *data, float *add){
    for(int i=0; i<n; i++){
        add[i] = data[i];
        for(int j=0; j<n; j++){
            if(i != j)
                add[i] += data[j];
        }
    }
}
```

なおC言語

課題2: 高速化例

メモリアクセスが多いほど、計算に時間がかかる

例: 各スレッドごとに変数を用意し、加算していく

add_dへのメモリアクセスは最後の1回だけにする

```
__global__ void kernel_B(const int n, float *ex_d, float *add_d){  
    int i = threadIdx.x + blockIdx.x * blockDim.x;  
    float add = ex_d[i];  
    for(int j = 0; j < n; j++) {  
        if(i != j) {  
            add += ex_d[j]; // 全データを加算  
        }  
    }  
    add_d[i] = add;  
}
```

コンパイラが良きに計らわないか・・・？

課題3：銀河シミュレーション

概要は以下の通り：

GPUで動作するプログラムを作成

さらにCPUで動作するプログラム(逐次版)も作成

星の数を増やして計算時間を測定、比較

CPUでの計算時間と比較

星の数は1000から10万程度まで変化させる

